

Documentation for Final Project CS 110

Section 1: What Your Project is Trying to Accomplish

Main goal

A roommate matching system that matches students to roommates based on their housing preferences.

This helps students find students based on their needs, habits, and compatibility.

Features:

- Collects detailed preferences through a questionnaire
- Calculates the compatibility and assigns a percentage to students.
- Matches students in different room types
- Adds the person who is filling the questionnaire into the database.
- It ranks the matches with detailed reasoning like similarities among the students.

Real-world problem it solves

Problem: Students struggle to find compatible roommates, leading to conflicts over sleep schedules, cleanliness, noise levels, and social preferences.

Solution: The program includes a questionnaire that analyzes compatibility.

Impact: Reduces roommate conflicts caused by incompatible roommates, improves the living experience, and helps students find great matches.

Section 2: How it accomplishes this:

Architecture:

- Data Storage - Uses the python file (' `roomates_database.py` ') to store the student profiles
- Data collection - The questionnaire collects the user data based on their preferences.
- Matching Algorithm - Calculates compatibility of individuals based on multiple factors
- Results display - Shows the top match among two individuals with percentage scores and the reasoning behind this score.

Technical Components

Database System:

- File Format - A python file which contains a list of dictionaries
- Structure - Each student's data is a dictionary with all the preferences
- Storage Method: Uses the python-native ' `pprint.pformat()` ' for formatting
- Operations:
 - Loads the existing database when the code starts
 - Creates a new database with 20 fictional students if none exists
 - Saves the users data who is filling out the form in the database so that the database is constantly updating
 - Uses a python standard library only (no external dependencies)

Questionnaire System:

Type of questions:

- Basic info - Year, major, room type, gender, honors status, accessibility, break housing
- Social and Lifestyle - Social level, noise preferences, environment, sleep schedule, tidiness, lifestyle match, tidiness, guest frequency
- Amenities - Kitchen importance, bathroom preference, campus and activities proximity.
- Community and Interests - Community type, shared interests, theme
- Special needs: Medical requirements, sensitivities, dietary preferences
- Priority Rankings - Ranks location, privacy, amenities and social atmosphere by importance.

Input Validation - Uses try/except blocks to handle invalid input

User-friendly - Accepts numbers or option text

Compatibility Scoring Algorithm

The ' `calculate_compatibility_score()` ' function evaluates matches based on scores:

Roommate compatibility factors:

- Room Type Match: +15 if same, -15 if different (critical for matching)
- Gender Compatibility: +10 if compatible
- Sleep Schedule: +20 if identical, +10 if one is flexible, -15 if conflicting
- Tidiness Level: +25 if identical, +10 if similar, -30 if very different
- Noise Level: +20 if matching, -25 if conflicting
- Social Level: +15 if matching, -15 if very different
- Guest Frequency: +15 if matching, -20 if major conflict
- Environment Preference: +15 if matching, -15 if conflicting
- Sensitivity Compatibility: +10 if no conflicts, -25 if pet conflicts
- Accessibility: Critical factor: -50 if required but not met

Scoring Process:

- Starts with a base score of 50
- Adds/subtracts points based on compatibility
- Ensures final score is between 0 and 100
- Generates detailed reasoning for each match

Matching system

- Function: ' `find_matches()` '
- Compares the user profile against everyone in the database
- Calculates score for each candidate
- Provides reasoning for the match
- Saves the user profile in the database for future matching

Results Display

- Shows compatibility score (1-100%)
- Calculates score for each candidates
- Sorts by the score (highest first)
- Returns the top 5 matches

Key programming Concepts used to accomplish everything:

Data structures

- Dictionaries - To store student data in key-value pairs
- Lists - Stores multiple students, options, priorities
- Tuples - Returns multiple values for candidates, scores and reasoning
- Sets - Track used names in order to avoid making duplicates

Control Flow

- If-elif-else: Validate input, check compatibility, handle different scenarios

- For loops: Iterate through the database, generate the students, process the matches
- While loops: Input validation till valid response is reached
- Nested Structures: Conditionals and loops were nested

Functions I used:

- `generate_fictional_student()` - Creates student profiles
- `create_database()` - Generates database
- `save_database()` - Writes to file
- `load_database()` - Reads from file
- `get_user_input()` - Validates input
- `collect_questionnaire()` - Gathers user preferences
- `calculate_compatibility_score()` - Computes match score
- `find_matches()` - Finds best matches
- `display_results()` - Shows results

Error Handling:

Try and except blocks - Handle file operations, input validation, keyboard interrupts.

File operations - Gracefully handle missing files and corrupted data

Input Validation: Ensures that responses are valid before proceeding.

Advanced Features (Had to refer to tutorials and documentation for):

List comprehensions: Generate student IDs, filter candidates:

Type castings: Converting strings into integers, ensuring proper types

Math modules: Use `floor()` for score calculations

pprint modules: Format data structures for a readable Python file output

File I/O: Read and write Python files to store roommate data. (I struggle with this; therefore, I had to refer to a tutorial even though we have done this in class.)

Section 3: How to Run your code

Prerequisites

- Python 3.6 or higher installed
- No external libraries needed (uses only Python standard library: ``random``, ``math``, ``typing``, ``os``, ``pprint``)

Step-by-Step Instructions

Step 1: Open Terminal/Command Prompt

- Mac/Linux:
 - Press ``Cmd + Space`` (Mac) or ``Ctrl + Alt + T`` (Linux) to open Terminal
 - Or search for "Terminal" in applications
- Windows:
 - Press ``Win + R``, type ``cmd`` and press Enter (for Command Prompt)
 - Or press ``Win + X`` and select "Windows PowerShell" or "Terminal"
 - Or search for "Command Prompt" or "PowerShell" in Start menu

Step 2: Navigate to Project Directory (All of this is inside the terminal)

For Mac/Linux:

```
cd "/Users/yourname/Documents/Final Project"
```

Or if your project is in a different location:

```
cd "/path/to/your/project/folder"
```

For Windows:

```
cd "C:\Users\YourUsername\Documents\Final Project"
```

Or if your project is on a different drive:

```
cd "D:\Projects\Final Project"
```

Tips:

- If your folder path has spaces, make sure to use quotes or escape characters
- You can also navigate by typing ``cd`` and then dragging the folder into the terminal window
- To check your current location, type ``pwd`` (Mac/Linux) or ``cd`` (Windows)

Step 3: Run the Program

For Mac/Linux:

```
python3 roommate_data.py
```

For Windows:

```
python roommate_data.py
```

Alternative commands if the above don't work:

- Try `py roommate_data.py` (Windows)
- Try `python3.9 roommate_data.py` or `python3.10 roommate_data.py` (if you have a specific version)
- If you get "command not found", make sure Python is installed and added to your PATH